

Bancos de dados relacionais espaciais

Disciplina: Programação aplicada à engenharia cartográfica

Maurício C. M. de Paulo - D.Sc.

18 de maio de 2026



- 1 Conceitos básicos de banco de dados geográficos
- 2 Extensões espaciais para bancos de dados
- 3 Prática com Postgresql/Postgis
- 4 Bancos de dados EDGV com DSG Tools
- 5 Execução remota de comandos SQL por bibliotecas Python
- 6 Exercícios complementares: PostGIS no QGIS Processing
- 7 Exercícios complementares: DuckDB



Banco de Dados Relacional (BDR):

- Dados organizados em tabelas (relações)
- Linhas → tuplas
- Colunas → atributos
- Chaves primárias e estrangeiras

Modelo baseado em:

- Álgebra relacional
- SQL (Structured Query Language)

Exemplo de SGBD:

- **PostgreSQL database system**
- **MySQL database system**

Dados em bancos relacionais são organizados em **tabelas**.

id	nome	população
1	Curitiba	1774000
2	Campinas	1214000
3	Recife	1490000

- **Colunas** representam atributos
- **Linhas** representam registros
- Cada tabela normalmente possui uma **chave primária (ID)**



SQL é a linguagem padrão para manipulação de bancos de dados relacionais.

Principais usos:

- Consultar dados
- Inserir novos registros
- Atualizar registros existentes
- Remover dados
- Definir estrutura de tabelas

Bancos que utilizam SQL:

- PostgreSQL
- MySQL
- SQLite (e Geopackage)
- DuckDB
- SQL Server



- 1 Conceitos básicos de banco de dados geográficos
- 2 Extensões espaciais para bancos de dados**
- 3 Prática com Postgresql/Postgis
- 4 Bancos de dados EDGV com DSG Tools
- 5 Execução remota de comandos SQL por bibliotecas Python
- 6 Exercícios complementares: PostGIS no QGIS Processing
- 7 Exercícios complementares: DuckDB

Objetivo: Adicionar suporte a dados geoespaciais em bancos relacionais.

Funcionalidades espaciais:

- Tipos geométricos:
 - Ponto, Linha, Polígono
- Índices espaciais:
 - Consultas rápidas por localização
- Funções espaciais:
 - Distância
 - Interseção
 - Buffer
 - Área
- Integração com GIS:
 - QGIS, GeoPandas, Leafmap

Padrões comun para colunas geométricas:

- OGC Simple Features
- WKT / WKB
- GeoJSON

Tecnologia	Extensão Espacial	Características
MySQL (SGBD)	Spatial Extensions	Suporte espacial integrado. Funções GIS básicas e índices espaciais.
PostgreSQL (SGDB)	PostGIS	Principal solução open-source GIS. Muito usado com QGIS e GeoPandas.
SQLite (em arquivo)	Spatialite / GeoPackage	Banco leve baseado em arquivo único. Muito usado em aplicações desktop e móveis.
DuckDB (em processo)	Spatial Extension	Foco em analytics e processamento geoespacial local de alta performance.

Exemplos de uso:

- PostGIS → servidores GIS
- GeoPackage → intercâmbio de dados
- DuckDB → análise geoespacial local



- 1 Conceitos básicos de banco de dados geográficos
- 2 Extensões espaciais para bancos de dados
- 3 Prática com Postgresql/Postgis**
- 4 Bancos de dados EDGV com DSG Tools
- 5 Execução remota de comandos SQL por bibliotecas Python
- 6 Exercícios complementares: PostGIS no QGIS Processing
- 7 Exercícios complementares: DuckDB

Recuperando nosso container de PostgreSQL+PostGIS da aula passada.

Iniciando o container

```
docker start postgis
```

Executa o terminal bash dentro do container

```
docker exec -it postgis bash
```

Criando um banco novo (no terminal do container)

```
createdb geodb -U postgres
```

Conectando ao banco (no terminal do container)

```
psql -U postgres -d geodb
```

Conectando ao banco (fora do terminal do container)

```
docker exec -it postgis psql -U postgres -d geodb
```

1. Alterar senha do usuário postgres:

```
ALTER USER postgres  
WITH PASSWORD 'nova_senha';
```

2. Criar usuário aluno:

```
CREATE USER aluno  
WITH PASSWORD 'senha123';
```

3. Permitir conexão ao banco geodb:

```
GRANT CONNECT ON DATABASE geodb TO aluno;
```

4. Dar permissões de leitura e escrita:

```
GRANT USAGE ON SCHEMA public TO aluno;  
  
GRANT SELECT, INSERT, UPDATE, DELETE  
ON ALL TABLES IN SCHEMA public  
TO aluno;
```

Conectando PostgreSQL no QGIS DB Manager



Passos básicos:

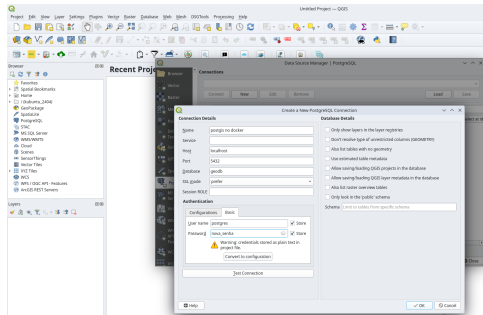
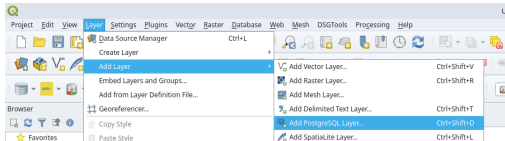
1 Selecionar:

- Camada - Adicionar camada
- Nova Conexão

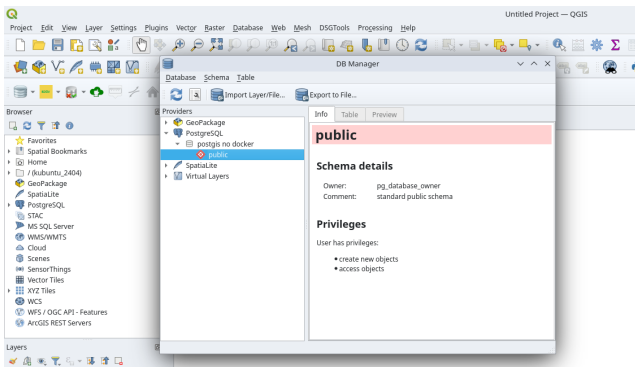
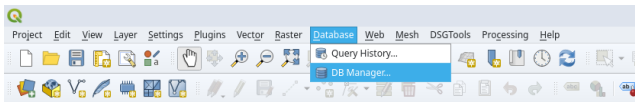
2 Informar:

- Host: localhost
- Porta (5432)
- Banco: geodb
- Usuário: postgres
- Senha: nova_senha

3 Testar conexão



Conectando PostgreSQL no QGIS



Selecionar o banco **geodb** e abrir a Janela SQL do DB Manager.

Criando a tabela cidades:

```
CREATE TABLE cidades (  
    id SERIAL PRIMARY KEY,  
    nome TEXT,  
    populacao INTEGER,  
    longitude REAL,  
    latitude REAL  
);
```

Obs.: o tipo serial é um inteiro que se autoincrementa. Usar id INTEGER PRIMARY KEY AUTOINCREMENT em Spatialite.

Inserindo registros iniciais:

```
INSERT INTO cidades  
(nome, populacao, longitude, latitude)  
VALUES  
('Rio de Janeiro', 6748000, -43.1729, -22.9068),  
('São Paulo', 12330000, -46.6333, -23.5505),  
('Belo Horizonte', 2531000, -43.9386, -19.9208);
```

A operação mais comum em SQL é a consulta de dados usando SELECT.

```
SELECT coluna1, coluna2  
FROM tabela  
WHERE condicao;
```

Componentes:

- **SELECT** - define as colunas retornadas
- **FROM** - define a tabela consultada
- **WHERE** - filtra os registros

Exemplo:

```
SELECT nome, populacao  
FROM cidades  
WHERE populacao > 3000000;
```

SQL também permite modificar os dados.

Inserir registros:

```
INSERT INTO cidades (nome, populacao, longitude, latitude)
VALUES ('Campinas', 1214000, -47.0608, -22.9056);
```

Atualizar registros:

```
UPDATE cidades
SET populacao = 1300000
WHERE nome = 'Campinas';
```

Remover registros:

```
DELETE FROM cidades
WHERE nome = 'Campinas';
```


BDRs podem ser estendidos para armazenar geometrias.

Exemplo:

- **PostGIS** extensão do PostgreSQL

Novo tipo de dado:

- GEOMETRY
- GEOGRAPHY

Permite armazenar:

- Pontos
- Linhas
- Polígonos

E executar consultas espaciais.



Cada feição espacial é composta por:

- Atributos (campos tabulares)
- Geometria (coluna espacial)

Estrutura típica:

id	nome	area	geom
1	Rio de Janeiro	43,696	MULTIPOLYGON(...)

A coluna `geom` armazena objetos como:

- POINT
- LINESTRING
- POLYGON



PostgreSQL:

- SGBD relacional cliente–servidor
- ACID completo
- Controle de concorrência (MVCC)
- Extensível

PostGIS:

- Extensão espacial do PostgreSQL
- Adiciona tipos GEOMETRY e GEOGRAPHY
- Implementa padrões OGC Simple Features

Resultado:

Banco relacional + engine espacial robusta



1. Tipo GEOMETRY

- Armazena POINT, LINESTRING, POLYGON, etc.
- Associado a um SRID (sistema de referência)

2. Funções Espaciais

3. Índices Espaciais

- GiST (Generalized Search Tree)
- Filtragem por bounding box
- Otimização de ST_Intersects, ST_Within, etc.

4. Modelo Cliente–Servidor

- Multiusuário
- Controle transacional
- Ideal para produção

	Shapefile	PostgreSQL + PostGIS	DuckDB + Spatial
Multiusuário	Não	Sim	Limitado
Transações ACID	Não	Sim	Sim
Cliente–Servidor	Não	Sim	Não
Instalação	Simples	Servidor dedicado	Arquivo único
Índice Espacial	.qix	GiST / SP-GiST	R-Tree
SQL Completo	Não	Sim	Sim
Escalabilidade	Baixa	Alta	Média

Posicionamento:

Shapefile → formato legado de troca.

PostgreSQL + PostGIS → infraestrutura robusta multiusuário.

DuckDB Spatial → banco analítico leve e embarcado.

Verificar se o PostGIS está carregado neste banco.

```
SELECT PostGIS_Version();
```

Caso não esteja instalado, vai retornar que a função não existe. Para instalar:

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

Inserindo uma coluna geometria à partir de colunas lat e lon:

```
-- Adiciona a coluna geom (geometria)
ALTER TABLE cidades
ADD COLUMN geom geometry(Point, 4326);

-- Preenche a geometria usando longitude e latitude
UPDATE cidades
SET geom = ST_SetSRID(
    ST_MakePoint(longitude, latitude),
    4326
);
```

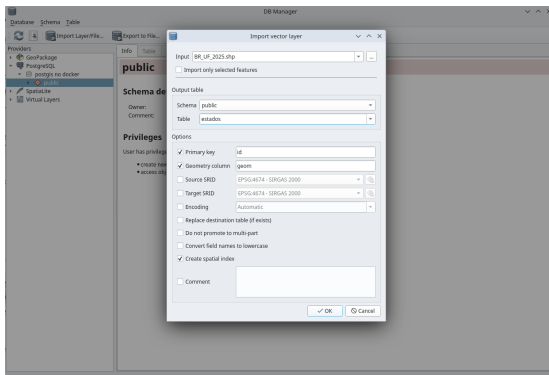
Agora é possível visualizar esta tabela no mapa do QGIS!

Inserindo dados geoespaciais no PostgreSQL pelo QGIS



Adicionar a camada vetorial no QGIS (baixando ou pelo URL)
https://geoftp.ibge.gov.br/organizacao_do_territorio/malhas_territoriais/malhas_municipais/municipio_2025/Brasil/BR_UF_2025.zip

- Acessar o DB Manager.
- Escolher a conexão ao PostGIS.
- Importar camada vetorial.
- Carregar no canvas.



Funções de agregação resumem dados.

```
SELECT COUNT(*)  
FROM estados;
```

Funções comuns:

- COUNT() - número de registros
- SUM() - soma
- AVG() - média
- MIN() - valor mínimo
- MAX() - valor máximo

Exemplo:

```
SELECT AVG(populacao)  
FROM cidades;
```


Agrupando cidades por faixa populacional:

```
SELECT "NM_REGIAO", count(*) AS quantidade_de_estados
FROM "BR_UF_2025"
GROUP BY "NM_REGIAO"
ORDER BY quantidade_de_estados;
```

Termos usados:

- **AS**: cria um alias (apelido) para uma coluna ou tabela. Muito útil para referenciar colunas geradas por funções (Ex.: Count(*)).
- **GROUP BY**: Agrupa valores iguais em uma coluna. Requer uma função de agregação depois (Ex.: Count()).
- **ORDER BY**: Ordena pelos valores de uma coluna.



Bancos relacionais permitem combinar dados de várias tabelas.

Exemplo de duas tabelas:

- cidades
- estados

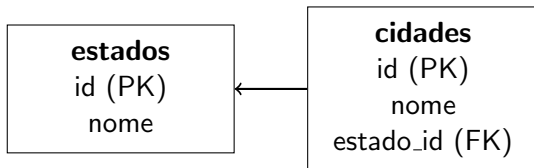
Consulta com **JOIN**:

```
SELECT c.nome, e.nome  
FROM cidades c  
JOIN estados e  
ON c.estado_id = e.id;
```

JOIN permite:

- combinar informações
- evitar duplicação de dados
- modelar relações entre entidades

Bancos relacionais conectam tabelas usando **chaves estrangeiras**.



cidade pertence a um estado

Exemplo de consulta combinando tabelas:

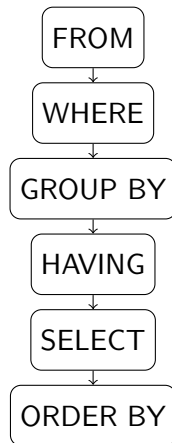
```
SELECT c.nome, e.nome
FROM cidades c
JOIN estados e
ON c.estado_id = e.id;
```

Embora a consulta seja escrita nesta ordem:

```
SELECT nome, populacao  
FROM cidades  
WHERE populacao > 100000  
ORDER BY populacao DESC;
```

- Primeiro o banco identifica a **tabela** (FROM)
- Depois aplica **filtros** (WHERE)
- Em seguida realiza **agrupamentos**
- Só então seleciona as **colunas finais**
- Por último ordena os resultados

O banco executa as operações aproximadamente nesta ordem:



Além de consultas tradicionais:

- SELECT
- JOIN
- GROUP BY

Existem funções espaciais:

- ST_Buffer()
- ST_Intersects()
- ST_Contains()
- ST_Area()

Exemplo conceitual:

Selecionar municípios que intersectam um rio.



Problemas comuns em consultas espaciais:

- Encontrar objetos que **intersectam** uma área
- Selecionar feições **próximas**
- Identificar objetos dentro de um **bounding box**

Sem índice espacial:

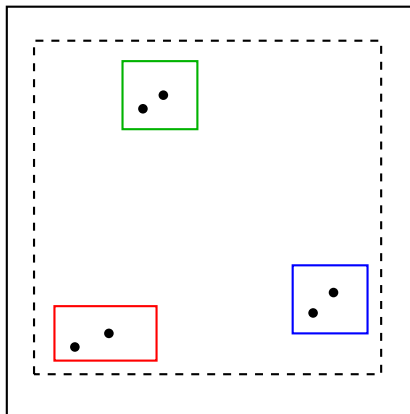
- Cada geometria precisa ser testada
- Complexidade aproximada: $O(n)$

Com índice espacial:

- Estrutura hierárquica de envelopes
- Redução drástica do número de testes geométricos
- Complexidade aproximada: $O(\log n)$

Ideia central:

- Primeiro filtrar por **bounding boxes**
- Depois aplicar a operação geométrica precisa



Objetos agrupados em **Minimum Bounding Rectangles (MBR)**

- Cada nó armazena o **retângulo mínimo** que cobre seus filhos
- Consultas percorrem apenas ramos relevantes

Exemplo em PostGIS:

Benefícios:

- Maurício C. M. de Paulo - D.Sc.

Objetivo: Melhorar desempenho de consultas espaciais e atributos.

Tabela: BR_UF_2025

Índice espacial (PostGIS):

```
CREATE INDEX idx_br_uf_geom  
ON BR_UF_2025  
USING GIST (geom);
```

Índice por código da UF:

```
CREATE INDEX idx_br_uf_cd_uf  
ON BR_UF_2025 (CD_UF);
```

Índice por área:

```
CREATE INDEX idx_br_uf_area  
ON BR_UF_2025 (AREA_KM2);
```

Carregar os arquivos abaixo no QGIS e importar para um banco de dados no PostgreSQL:

- 1 ja_visitei.csv
- 2 BR_UF_2025.zip (shapefile)

Também será necessária a tabela cidades, construída anteriormente.

Criar uma tabela com o resultado da operação de join de ja_visitei e "BR_UF_2025"

Carregar os arquivos abaixo no QGIS e importar para um banco de dados no PostgreSQL:

- 1 ja_visitei.csv
- 2 BR_UF_2025.zip (shapefile)

Também será necessária a tabela cidades, construída anteriormente.

Criar uma tabela com o resultado da operação de join de ja_visitei e "BR_UF_2025"

```
CREATE table ja_visitei_do_brasil as
select br.* , ja_visitei.visitei
from "BR_UF_2025" as br
join ja_visitei
on ja_visitei.nome = br.name;
```



Selecionar os estados que contém os pontos da tabela cidades

Selecionar os estados que contém os pontos da tabela cidades

```
SELECT *  
FROM "BR_UF_2025" AS estados, cidades  
WHERE st_intersects( estados.geom, st_setsrid(cidades.geom, 4674));
```

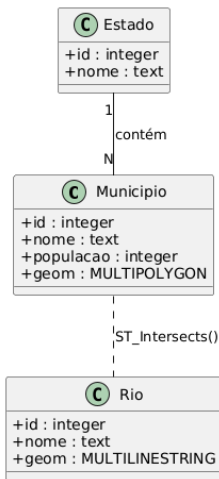
Adicionar uma coluna uf na tabela cidades, com o resultado da operação de interseção

Selecionar os estados que contém os pontos da tabela cidades

```
SELECT *  
FROM "BR_UF_2025" AS estados, cidades  
WHERE st_intersects( estados.geom, st_setsrid(cidades.geom, 4674));
```

Adicionar uma coluna uf na tabela cidades, com o resultado da operação de interseção

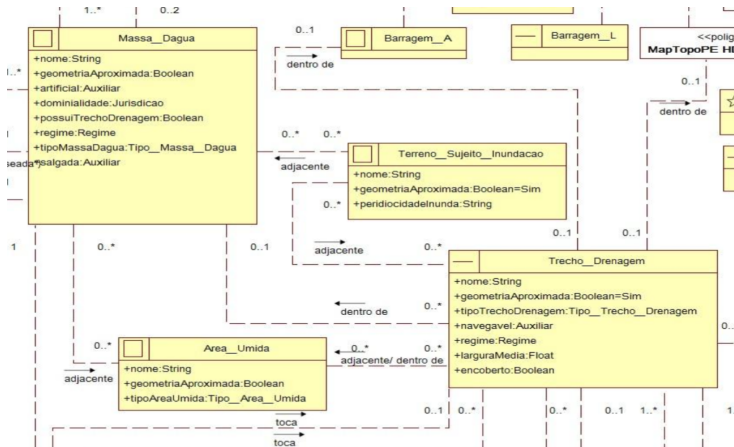
```
ALTER TABLE cidades  
ADD COLUMN uf VARCHAR(2);  
  
UPDATE cidades  
SET uf = estados."SIGLA_UF"  
FROM "BR_UF_2025" AS estados  
WHERE st_intersects( estados.geom, st_setsrid(cidades.geom, 4674));
```



Exercícios: Implementar a EDGV no PostGIS



Implementar usando SQL.



EDGV 3.0

Modelos conceituais da EDGV 3.0 (Anexos)



- 1 Conceitos básicos de banco de dados geográficos
- 2 Extensões espaciais para bancos de dados
- 3 Prática com Postgresql/Postgis
- 4 Bancos de dados EDGV com DSG Tools**
- 5 Execução remota de comandos SQL por bibliotecas Python
- 6 Exercícios complementares: PostGIS no QGIS Processing
- 7 Exercícios complementares: DuckDB

Abrir no GitHub:

edgv3.sql

edgv3_pro.sql

Conversor de modelagens (Nova implementação): edgv3.sql

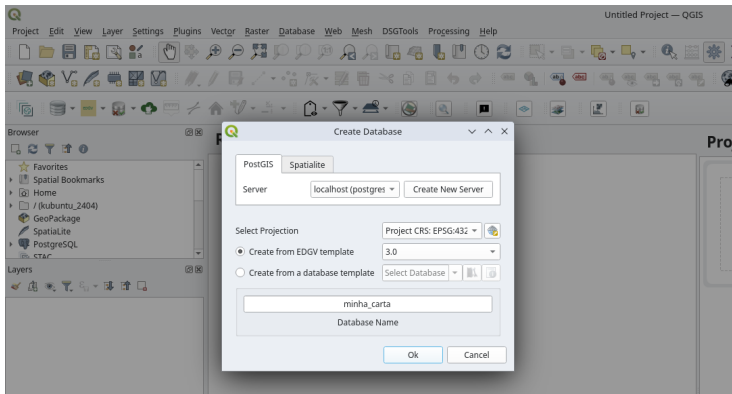


```
14273
14274 CREATE TABLE edgv.hid_trecho_drenagem_l(
14275     id uuid NOT NULL DEFAULT uuid_generate_v4(),
14276     nome varchar(255),
14277     geometriaaproximada boolean NOT NULL,
14278     tipotrechodrenagem smallint NOT NULL,
14279     navegavel smallint NOT NULL,
14280     larguramedia real,
14281     regime smallint NOT NULL,
14282     encoberto boolean NOT NULL,
14283     observacao VARCHAR(255),
14284     geom geometry(MultiLineString, [epsg]),
14285     CONSTRAINT hid_trecho_drenagem_l_pk PRIMARY KEY (id)
14286     WITH (FILLFACTOR = 80)
14287 )#
14288 CREATE INDEX hid_trecho_drenagem_l_geom ON edgv.hid_trecho_drenagem_l USING gist (geom)#
14289
14290 ALTER TABLE edgv.hid_trecho_drenagem_l OWNER TO postgres#
14291
14292 ALTER TABLE edgv.hid_trecho_drenagem_l
```

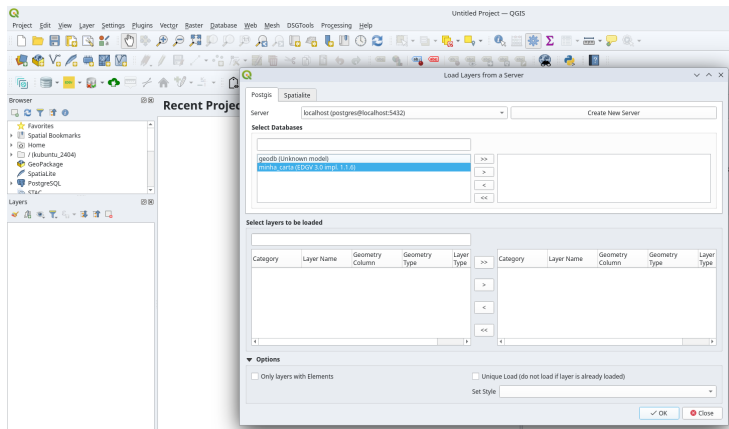
Criação do banco de dados em conformidade com uma EDGV



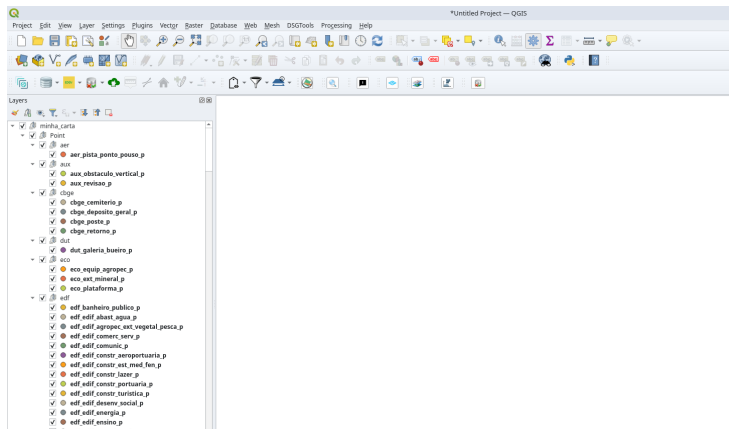
Crie uma nova conexão (não usa a padrão do QGIS).
Para ET-EDGV 3.0 ou ET-EDGV 2.1.3:



Selecionar todas as camadas do banco:



Carregando camadas do banco de dados EDGV



- 1 Conceitos básicos de banco de dados geográficos
- 2 Extensões espaciais para bancos de dados
- 3 Prática com Postgresql/Postgis
- 4 Bancos de dados EDGV com DSG Tools
- 5 Execução remota de comandos SQL por bibliotecas Python**
- 6 Exercícios complementares: PostGIS no QGIS Processing
- 7 Exercícios complementares: DuckDB



Integração Python + PostgreSQL

Existem diversas bibliotecas Python que permitem:

- Conectar ao PostgreSQL
- Executar comandos SQL remotamente
- Ler e escrever tabelas
- Trabalhar com dados espaciais do PostGIS

Execução remota de SQL

- O código Python envia comandos SQL ao servidor
- O processamento ocorre no PostgreSQL/PostGIS
- Consultas utilizam índices e otimizações do banco

Vantagem: Grandes volumes de dados podem ser processados diretamente no servidor.

```
from PyQt5.QtSql import QSqlDatabase, QSqlQuery

db = QSqlDatabase.addDatabase("QPSQL")

db.setHostName("localhost")
db.setDatabaseName("geodb")
db.setUserName("postgres")
db.setPassword("postgres")

db.open()

query = QSqlQuery()

query.exec(""" SELECT nome, populacao FROM cidades """)

while query.next():
    print(
        query.value(0),
        query.value(1)
    )
```




```
from osgeo import ogr

conn_str = (
    "PG:host=localhost "
    "dbname=geodb "
    "user=postgres "
    "password=postgres"
)

ds = ogr.Open(conn_str)

sql = """ SELECT nome, populacao FROM cidades """

layer = ds.ExecuteSQL(sql)

for feat in layer:
    print(
        feat.GetField("nome"),
        feat.GetField("populacao")
    )
```



```
import geopandas as gpd

from sqlalchemy import create_engine

engine = create_engine(
    "postgresql://postgres:"
    "postgres@localhost/geodb"
)

sql = """ SELECT * FROM cidades """

gdf = gpd.read_postgis(
    sql,
    engine,
    geom_col="geom"
)

print(gdf.head())
```



```
import psycopg2

conn = psycopg2.connect(
    host="localhost",
    dbname="geodb",
    user="postgres",
    password="postgres"
)

cur = conn.cursor()

sql = """
SELECT nome, populacao
FROM cidades
"""

cur.execute(sql)

for row in cur.fetchall():
    print(row)
```

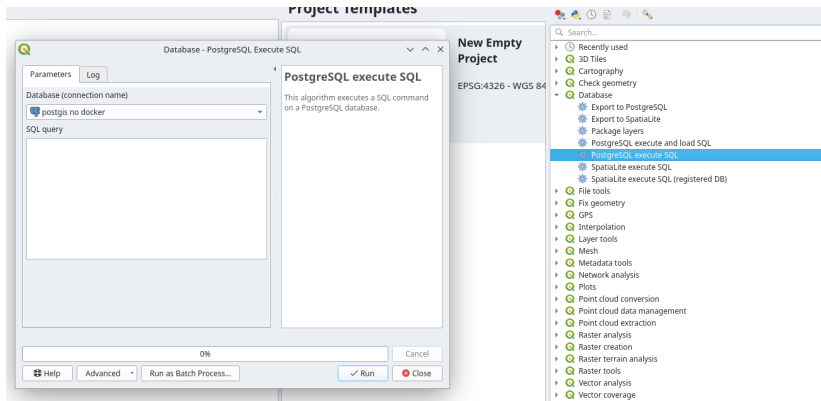


```
from qgis.core import QgsProviderRegistry
import psycopg2
# Nome da conexão configurada no QGIS
connection_name = 'postgis no docker'
# Obtém metadados do provider PostgreSQL
metadata = QgsProviderRegistry.instance().providerMetadata('postgres')
# Recupera a conexão salva no QGIS
conn = metadata.findConnection(connection_name, False)
uri = conn.uri() # Parâmetros da conexão
# Conecta no PostgreSQL
pg_conn = psycopg2.connect(uri)
cursor = pg_conn.cursor()
# SQL que será executado
sql = """ create table teste as select * from cidades; """
cursor.execute(sql)
pg_conn.commit()
print('SQL executado com sucesso.')
# Importante: Fechar a conexão.
cursor.close()
pg_conn.close()
```



- 1 Conceitos básicos de banco de dados geográficos
- 2 Extensões espaciais para bancos de dados
- 3 Prática com Postgresql/Postgis
- 4 Bancos de dados EDGV com DSG Tools
- 5 Execução remota de comandos SQL por bibliotecas Python
- 6 Exercícios complementares: PostGIS no QGIS Processing**
- 7 Exercícios complementares: DuckDB

Exercícios complementares: PostGIS no QGIS Processing





- 1 Conceitos básicos de banco de dados geográficos
- 2 Extensões espaciais para bancos de dados
- 3 Prática com Postgresql/Postgis
- 4 Bancos de dados EDGV com DSG Tools
- 5 Execução remota de comandos SQL por bibliotecas Python
- 6 Exercícios complementares: PostGIS no QGIS Processing
- 7 Exercícios complementares: DuckDB**

DuckDB é um banco de dados analítico embutido (embedded OLAP).
OLAP (Online Analytical Processing)

Características principais:

- Arquivo único (.duckdb)
- Sem servidor (in-process)
- Execução vetorizada (vectorized engine)
- Otimizado para leitura intensiva (OLAP)

Casos de uso típicos:

- Análise local de grandes datasets
- Ciência de dados
- Processamento de Parquet
- Integração com Python / R



1. Embedded Database

- Executa dentro do processo Python
- Não há daemon ou serviço separado

2. Columnar Storage

- Armazenamento orientado a colunas
- Ideal para agregações e scans analíticos

3. Integração com formatos modernos

- Parquet (leitura direta)
- CSV
- Arrow
- Extensão Spatial (tipos GEOMETRY)

4. OLAP vs OLTP

- Excelente para consultas analíticas
- Não projetado para alta concorrência transacional



Abrir no colab

O **Pixi** permite criar ambientes isolados e instalar ferramentas rapidamente.

1. Criar um novo projeto Pixi

```
pixi init pixi_env  
cd pixi_env
```

2. Instalar o DuckDB CLI

```
pixi add duckdb-cli
```

3. Executar o DuckDB

```
pixi run duckdb
```

Verificando a instalação

```
SELECT version();
```

- Ambiente reproduzível
- Sem necessidade de instalar o banco globalmente
- Ideal para aulas e projetos de análise de dados

DuckDB permite consultar dados diretamente de arquivos online usando SQL.

Exemplo usando um dataset público de cidades:

```
INSTALL httpfs;  
LOAD httpfs;  
  
CREATE TABLE cidades AS  
SELECT *  
FROM read_csv_auto(  
  'https://raw.githubusercontent.com/datasets/world-cities/master/data/world-cities.csv'  
);
```

Agora podemos executar consultas SQL normalmente:

```
SELECT name, country  
FROM cidades  
WHERE country = 'Brazil'  
LIMIT 10;
```

Instalação (uma vez):

```
pixi add duckdb
```

Exemplo em Python (parte 1):

```
import duckdb

# conecta (arquivo será criado se não existir)
con = duckdb.connect("dados.duckdb")

# instala e carrega extensão espacial
con.execute("INSTALL spatial;")
con.execute("LOAD spatial;")
```

```
# cria tabela espacial
con.execute("""
    CREATE TABLE municipios AS
    SELECT
        1 AS id,
        'Município A' AS nome,
        ST_GeomFromText(
            'POLYGON((-48 -15, -48 -16, -47 -16, -47 -15, -48 -15))'
        ) AS geom;
""")

# consulta área
result = con.execute("""
    SELECT nome,
    ST_Area(geom) AS area
    FROM municipios;
""").fetchall()

print(result)
```

```
import requests, zipfile, io
import duckdb

# 1.Download (Natural Earth - países 110m, leve)
url = "https://naturalearth.s3.amazonaws.com/110m_cultural/ne_110m_admin_0_countries.zip"
response = requests.get(url)
response.raise_for_status()

# 2.Extrai zip em memória
with zipfile.ZipFile(io.BytesIO(response.content)) as z:
    z.extractall("ne_countries")

# 3.Conecta ao DuckDB
con = duckdb.connect("world.duckdb")
con.execute("INSTALL spatial;")
con.execute("LOAD spatial;")
```

```
# 4. Lê shapefile direto via GDAL
con.execute("""
    CREATE TABLE countries AS
    SELECT * FROM ST_Read(
        'ne_countries/ne_110m_admin_0_countries.shp');
""")

# 5. Calcula área (m² - CRS 4326 → 3857)
result = con.execute("""
    SELECT NAME,
    ST_Area(ST_Transform(geom, 3857)) AS area
    FROM countries
    ORDER BY area DESC
    LIMIT 5;
""").fetchall()

for row in result:
    print(row)
```